

机器学习基础（二）

目 录

1 引言：为什么需要深度学习系统优化	3
2 硬件基础：深度学习的算力底座	3
2.1 CPU vs GPU	3
2.2 GPU 编程生态：CUDA 与加速库	3
2.3 TPU：专用张量加速	4
3 剪枝：让模型变稀疏	4
3.1 为什么需要剪枝	4
3.2 剪枝的直觉与形式	4
3.3 剪枝流程	4
3.4 剪枝不等于加速	5
4 量化：用低精度表示数值	5
4.1 为什么需要量化	5
4.2 量化公式	5
4.3 低精度数据类型谱系	5
4.4 对称 vs 非对称量化	6
4.5 PTQ vs QAT	6
5 并行策略：把模型拆到多张卡	7
5.1 为什么需要并行	7
5.2 数据并行	7
5.3 ZeRO：消除数据并行的显存冗余	7
5.4 流水线并行	7
5.5 张量并行	8
5.6 序列/上下文并行	8
5.7 专家并行	8
5.8 混合并行：4D Parallelism	8
6 显存优化技术	9
6.1 梯度累积	9
6.2 梯度检查点	9
7 推理加速	9
7.1 KV Cache	9
7.2 KV Cache 压缩	10
7.3 Speculative Decoding	10
7.4 Continuous Batching	10
7.5 算子融合与 FlashAttention	10

8 实战：对称量化代码示例	11
9 本章你将学会	11
10 要点速查	11
11 小结	12

1 引言：为什么需要深度学习系统优化

上一章我们学习了深度学习的基础：模型、损失、优化器，以及 CNN、RNN、Transformer 等网络结构。但你有没有想过，当一个模型大到单张 GPU 都装不下时，该怎么办？

Llama 3 这样的模型有数百亿甚至数千亿参数，远超单卡显存容量。与此同时，训练和推理的算力需求增长速度远快于硬件发展。我们面临的现实是：模型越来越大，硬件有限，必须想办法让模型在有限硬件上“跑得起来、跑得快、跑得省”。

直觉 这一章的核心问题就是：给定一个已经训练好的大模型，如何在有限的硬件资源下高效地完成训练和推理？这不仅是工程问题，更是一门系统优化的艺术。

本章覆盖四大主题：**硬件基础**（深度学习依赖什么算力）、**剪枝**（让模型变稀疏）、**量化**（用低精度表示数值）、**并行策略**（把模型拆到多张卡），以及推理加速技术（KV Cache、投机解码、算子融合）。

2 硬件基础：深度学习的算力底座

2.1 CPU vs GPU

直觉 不妨这样想：CPU 像几个顶尖教授，每个都能处理极其复杂的任务，但人数少，总吞吐有限；GPU 像成千上万个本科生，每人只能做简单计算，但人多力量大，并行处理海量数据时效率极高。

1. **CPU** (Central Processing Unit): 少量强核心，擅长低延迟的复杂逻辑串行计算
2. **GPU** (Graphics Processing Unit): 大量弱核心，擅长高吞吐的大规模并行矩阵计算

深度学习训练的本质是大规模矩阵运算（前向传播、反向传播都是矩阵乘法），因此主要依赖 GPU。

2.2 GPU 编程生态：CUDA 与加速库

CUDA (Compute Unified Device Architecture) 是 NVIDIA GPU 的并行编程框架。直接手写 CUDA 代码虽然灵活，但开发成本高。NVIDIA 在 CUDA 之上封装了多个优化算子库：

1. **cuBLAS**: 线性代数运算（矩阵乘法等）
2. **cuFFT**: 快速傅里叶变换
3. **cuDNN**: 深度学习专用算子（卷积、池化、归一化等）

手写 CUDA 与调用 cuDNN 库在性能上有显著差距。cuDNN 针对每种 GPU 架构做了深度优化，普通开发者很难超越。

其他 GPU 编程方案还有 **OpenCL**（跨平台，但在 NVIDIA 硬件上通常慢约 30%）和 **AMD ROCm / HIP**（AMD GPU 的 CUDA 替代方案）。

2.3 TPU：专用张量加速

TPU (Tensor Processing Unit) 是 Google 专为张量计算设计的专用芯片 (如 TPU v4)。它通过 **脉动阵列** (Systolic Array) 高效执行矩阵乘法, 多个 TPU 可组成 **Pod** 集群提供大规模算力。

CPU、GPU 是通用处理器, TPU 则是针对矩阵乘法“量身定制”的专用芯片 (ASIC)。专用意味着高效, 但也意味着灵活性低。

3 剪枝：让模型变稀疏

3.1 为什么需要剪枝

研究表明, 深度学习模型通常是 **过参数化** (Over-parameterized) 的: 模型中有很多参数对最终输出贡献很小, 甚至可以忽略。**剪枝** (Pruning) 就是删除这些不重要的参数 (权重), 使权重矩阵变稀疏, 从而减少计算量和显存占用。

直觉 想象一棵枝繁叶茂的大树, 有些枝条已经枯死却不影响整棵树的生长。剪枝就是剪掉这些“枯枝”, 让树更精简但依然健康。

3.2 剪枝的直觉与形式

剪枝的核心思路是: 找到“不重要”的权重, 将其置零。如何衡量重要性? 常见有两种方法:

1. **基于幅值的剪枝** (Magnitude-based Pruning): 按权重绝对值大小决定是否剪掉, 幅值小的被认为不重要
2. **基于损失最小化的剪枝** (Loss-based Pruning): 以剪枝后模型损失增长最小为目标选择保留哪些参数

例 假设某层权重为 $\begin{pmatrix} 0.01 & 0.82 & -0.03 \\ 0.45 & -0.91 & 0.005 \\ -0.02 & 0.67 & 0.38 \end{pmatrix}$ 。

按幅值从小到大排序: 0.005, 0.01, 0.02, 0.03, 0.38, 0.45, 0.67, 0.82, 0.91。

若设稀疏比例为 50% (剪掉 9 个中的 4 个最小的), 则 0.005, 0.01, 0.02, 0.03 被置零:

$$\begin{pmatrix} 0 & 0.82 & 0 \\ 0.45 & -0.91 & 0 \\ 0 & 0.67 & 0.38 \end{pmatrix} \quad (3.1)$$

可以看到, 剪枝后矩阵中出现了很多零, 这就是 **稀疏性** (Sparsity)。

3.3 剪枝流程

完整的剪枝流程为:

确定稀疏比例 (Sparsity Ratio) $\rightarrow$ 剪枝 $\rightarrow$ 微调 (Finetuning) 恢复精度。

直觉 剪枝会损失精度，因此剪完后通常需要再微调，让剩余权重重新适应以恢复性能。这就像修剪树木后需要施肥，让树恢复生机。

3.4 剪枝不等于加速

关键注意：剪枝不等于加速！非结构化（细粒度）稀疏需要特殊硬件支持才能真正加速。否则剪完后仍是稠密计算，速度不会提升。多数 GPU 只支持结构化（块级）稀疏的加速。

直觉 原因在于，稀疏矩阵中零元素的位置是随机的，GPU 的矩阵乘法内核无法高效跳过这些零。只有当零元素呈现规律性排列（如 2:4 结构化稀疏）时，硬件才能利用稀疏性加速。

4 量化：用低精度表示数值

4.1 为什么需要量化

上一节讲到，剪枝产生的稀疏矩阵不硬件友好，计算加速需要特殊硬件支持。**量化** (Quantization) 提供了另一条路：用低精度数据类型（如 INT8、FP4）表示原本高精度（FP32）的数值。

直觉 剪枝和量化的根本区别：剪枝改变矩阵的稀疏结构（部分值变零），量化改变数值的精度（所有值保留但精度降低）。量化后矩阵仍然是稠密的，仍可用高效的稠密矩阵乘（GEMM）计算，只是精度更低。因此量化比剪枝更容易获得实际加速。

量化的好处：容易减少显存用量，仍然是稠密 GEMM（只是精度更低），不需要特殊硬件支持。

4.2 量化公式

量化的核心公式：

$$r = (q - Z) \times S \quad (4.1)$$

其中 r 是实值 (Real Value)， q 是量化整数值 (Quantized Value)， Z 是 **零点** (Zero Point)， S 是 **缩放因子** (Scale)。

反量化时： $q = \text{round}(\frac{r}{S}) + Z$ 。

4.3 低精度数据类型谱系

从高精度到低精度的数据类型谱系：

类型	指数位	尾数位	特点
FP32	8	23	训练默认精度
FP16	5	10	混合精度训练常用
BF16	8	7	动态范围大，不易溢出
FP8 (E4M3)	4	3	前向推理常用

FP8 (E5M2)	5	2	动态范围更大
INT8	-	-	整数精度，推理加速
FP4 / INT4	-	-	极限压缩，大模型量化趋势

直觉 精度越低，能表示的数值范围和分辨率越有限，但显存占用和计算速度越优。量化的艺术在于：在精度损失可接受的范围内，尽可能降低数据位宽。

4.4 对称 vs 非对称量化

1. **对称量化** (Symmetric Quantization): 量化范围关于零点对称, $Z = 0$, 计算更简单
2. **非对称量化** (Asymmetric Quantization): $Z \neq 0$, 能更充分利用量化范围但计算更复杂

例 对权重 $r = \begin{pmatrix} -1.2 & 0.5 \\ -0.3 & 2.1 \end{pmatrix}$ 做对称量化到 INT8 (范围 $[-128, 127]$)。

第 1 步: 求缩放因子。 $S = \frac{\max(|r|)}{127} = \frac{2.1}{127} \approx 0.0165$, $Z = 0$ 。

第 2 步: 量化。 $q = \text{round}(\frac{r}{S})$:

- $q_{1,1} = \text{round}(\frac{-1.2}{0.0165}) = \text{round}(-72.7) = -73$
- $q_{1,2} = \text{round}(\frac{0.5}{0.0165}) = \text{round}(30.3) = 30$
- $q_{2,1} = \text{round}(\frac{-0.3}{0.0165}) = \text{round}(-18.2) = -18$
- $q_{2,2} = \text{round}(\frac{2.1}{0.0165}) = \text{round}(127.3) = 127$

第 3 步: 反量化验证。 $\hat{r} = q \times S$:

- $\hat{r}_{1,1} = -73 \times 0.0165 = -1.2045$ (原值 -1.2 , 误差 0.0045)
- $\hat{r}_{2,2} = 127 \times 0.0165 = 2.0955$ (原值 2.1 , 误差 0.0045)

量化误差很小，这就是量化有效的根本原因。

4.5 PTQ vs QAT

量化有两种策略:

PTQ (Post-Training Quantization, 训练后量化): 在模型训练完成后直接压缩, 成本低。代表方法有 **GPTQ** 和 **AWQ**。

Outlier 问题: PTQ 的主要痛点是少数异常大的权重 (Outlier) 会撑大量化范围, 导致大多数正常权重的精度骤降。解决方法包括 *Range Clipping* (范围裁剪)、混合精度 (Mix-precision, 对 outlier 保留高精度) 和 *SmoothQuant* (将激活的异常值平滑迁移到权重上)。

QAT (Quantization-Aware Training, 量化感知训练): 在训练过程中就模拟量化误差, 让模型提前适应低精度。精度通常优于 PTQ, 但训练成本更高。

直觉 PTQ 像是“考完试再减肥”, 低成本但可能影响发挥; QAT 像是“训练时就带着负重跑”, 适应了低精度后正式比赛时表现更好。

5 并行策略：把模型拆到多张卡

5.1 为什么需要并行

当模型大到单卡装不下时，必须将计算分散到多张卡上。并行策略就是研究“如何切分模型和数据”的学问。切分方式不同，通信开销和显存效率也截然不同。

主要的并行策略包括：

1. 数据并行 (Data Parallelism)
2. 流水线并行 (Pipeline Parallelism)
3. 张量并行 (Tensor Parallelism)
4. 序列/上下文并行 (Sequence/Context Parallelism)
5. 专家并行 (Expert Parallelism)

5.2 数据并行

每张卡保留完整模型副本，各自处理不同的数据批次。

1. 显存：每张卡存完整模型
2. 通信：只在前后阶段同步梯度
3. 吞吐：线性扩展（卡数翻倍，吞吐翻倍）
4. 延迟：单次推理延迟几乎不变

直觉 数据并行像多家工厂各自生产同一种产品但接不同订单。产量随工厂数线性增长，但单个订单的交货时间不变。

数据并行只提升吞吐量，不降低单次延迟。这是它与模型并行的根本区别。

5.3 ZeRO：消除数据并行的显存冗余

数据并行中每张卡都存完整模型，造成显存冗余。**ZeRO** (Zero Redundancy Optimizer) 通过分阶段消除这些冗余：

1. **Stage 1**：分片优化器状态 (Optimizer States)
2. **Stage 2**：在 Stage 1 基础上再分片梯度 (Gradients)
3. **Stage 3**：在 Stage 2 基础上再分片模型参数 (Parameters)

直觉 Stage 越高，显存越省，但通信越多。Stage 3 把模型参数也切分了，每张卡只存一部分参数，计算时需要临时从其他卡收集 (All-Gather) 完整参数。因此 Stage 3 显存最省但通信最重，并非总最优，需根据模型大小与通信带宽权衡。

5.4 流水线并行

按层把模型切分到多张卡上，每张卡负责若干层，数据像流水线一样逐 stage 传递。

1. 显存：每张卡只存部分层
2. 通信：每个 stage 结束后传递中间激活
3. 吞吐：可扩展

4. 延迟：会增加

直觉 流水线并行像工厂流水线：产品经过一道道工序，每台机器只负责一道工序。问题是前一台机器在处理时，后面的机器可能空闲，产生“气泡”（Bubble）。

为了填补气泡，需要将数据拆成多个**微批次**（Micro-batch），让不同卡同时处理不同批次。常见调度策略：

1. **GPipe**：先做完所有前向，再做所有反向。气泡较多
2. **1F1B**（One Forward One Backward）：一个前向后紧跟一个反向，减少气泡
3. **Interleaved**：将每个 pipeline stage 分成多个 chunk，优先执行反向，进一步减少气泡
4. **DualPipe**（DeepSeek）：双向流水线，前向和反向同时进行

5.5 张量并行

把单个张量或矩阵切分到多张卡上，几乎每个算子都需要通信。

1. **显存**：每张卡只存张量的一部分
2. **通信**：几乎每个操作都需要
3. **吞吐**：可扩展
4. **延迟**：受通信带宽限制

直觉 张量并行把一个大矩阵乘法拆成多个小矩阵乘法分给不同卡。每步计算完都需要 All-Reduce 合并结果，因此通信开销最大，适合高带宽互联（如 NVLink）的卡内或机内并行。

Megatron 首先提出了 1D 张量并行方案，后续还有 2D 方案进一步提升效率。

5.6 序列/上下文并行

将激活（Activation）按序列维度分片到多张卡上，减少长序列推理时的显存压力。通信只在某些操作后发生。

在长上下文推理（如 100K token）场景下，激活的显存占用远超模型参数，序列并行是关键手段。

5.7 专家并行

专家并行（Expert Parallelism）配合 **MoE**（Mixture of Experts，混合专家模型）使用：将不同的专家（FFN）分布到不同卡上，每个 token 只激活部分专家，通信发生在专家 FFN 计算前后。

5.8 混合并行：4D Parallelism

Llama 3 训练中同时使用了四种并行：数据 × 流水线 × 张量 × 专家，称为 **4D Parallelism**。不同并行策略各有优劣，实际部署中需要根据模型结构、硬件拓扑和通信带宽组合使用。

策略	显存	通信	吞吐	延迟
----	----	----	----	----

数据并行	完整模型	低	线性扩展	不变
ZeRO Stage 3	最省	最高	可扩展	增加
流水线并行	部分层	中	可扩展	增加
张量并行	部分张量	最高	可扩展	通信受限
序列并行	分片激活	中	可扩展	通信受限
专家并行	分片专家	中	可扩展	视路由而定

6 显存优化技术

6.1 梯度累积

当显存不足以容纳大 batch 时，可以将一个大 batch 拆成多个小 batch，分别前向后累积梯度，最后统一更新。等效于增大 batch size，减少优化器迭代步数。

```
optimizer.zero_grad()
for i, micro_batch in enumerate(data.split(8)):
    loss = model(micro_batch)
    loss = loss / 8
    loss.backward()
optimizer.step()
```

逐行批注：

1. `optimizer.zero_grad()`：清空上一步梯度缓存
2. `data.split(8)`：将大 batch 拆成 8 个微批次
3. `loss / 8`：缩放损失，确保梯度总和等价于大 batch
4. `loss.backward()`：反向传播，梯度自动累积到 `.grad` 中
5. `optimizer.step()`：累积完毕后一次性更新参数

6.2 梯度检查点

梯度检查点（Gradient Checkpointing）在前向后丢弃中间激活值以节省显存，反向传播时重新计算这些激活。

直觉 正常训练时，前向传播保存所有中间激活供反向传播使用，显存占用大。梯度检查点选择丢弃部分激活，反向时重算。这是用时间换空间的经典 trade-off：速度慢约 20%-30%，但显存大幅减少。

当显存不足导致 OOM（Out of Memory）时，梯度检查点是首选的应急手段。

7 推理加速

7.1 KV Cache

自回归生成（如 GPT 逐 token 生成）中，每生成一个新 token 都需要用到之前所有 token 的 Key 和 Value。如果每次都重新计算，效率极低。

KV Cache 的思路是：把历史 token 的 Key/Value 缓存下来，新 token 只需计算自己的 K/V 并追加到缓存中，避免重复计算。

直觉 想象你在读一本书，每读一页就把关键信息记在笔记本上。下次需要回顾时翻笔记本即可，不用从头再读一遍。KV Cache 就是这本“笔记本”。

在长上下文推理（如 32K 或 100K token）时，KV Cache 的显存占用远超模型参数本身，成为显存大头。必须通过压缩来控制。

7.2 KV Cache 压缩

KV Cache 压缩的三种主要方法：

1. **驱逐** (Evicting)：丢弃不重要的 token 的 KV 缓存
2. **合并** (Merging)：将相邻层的 KV Cache 合并以减少冗余
3. **量化** (Quantization)：用低精度存储 KV Cache

7.3 Speculative Decoding

投机解码 (Speculative Decoding) 用一个小模型快速“草拟”几个候选 token，再用大模型一次性验证。如果小模型猜对了，就跳过大模型的逐步生成；如果猜错了，大模型从错误处重新生成。

直觉 就像写文章时先让助手打个草稿，你再快速审阅修改。草稿大部分正确时效率极高；大部分错误时，不如你自己写。

7.4 Continuous Batching

传统批处理要求同一批次的所有请求同时到达、同时完成。**Continuous Batching**（连续批处理）动态组装批次：新请求随时加入，已完成的请求随时退出，不需要等待整个批次完成。

直觉 传统批处理像公交车：满员发车，到站所有人一起下车。连续批处理像传送带：东西随时放上去，到了就取走，传送带永不停歇。这大大提升了 GPU 利用率。

7.5 算子融合与 FlashAttention

算子融合 (Operator Fusion) 将多个小算子合并为一个大算子，减少中间结果的读写（访存开销）。

FlashAttention 是 Attention 计算的高效融合实现，通过分块计算和减少 HBM 读写来大幅加速 Attention 运算，同时降低显存占用。它已成为主流大模型训练与推理的标准组件。

直觉 Attention 的朴素实现需要把 $N \times N$ 的注意力矩阵完整写入 HBM 再读回，访存开销巨大。FlashAttention 将计算分块，让中间结果尽量留在 SRAM 中，避免频繁访存。这就像做菜时把所有食材一次性切好再炒，比分批切分批炒更高效。

8 实战：对称量化代码示例

下面用 PyTorch 演示对称量化与反量化的完整流程：

```
import torch

def symmetric_quantize(x, n_bits=8):
    q_max = 2 ** (n_bits - 1) - 1
    scale = x.abs().max() / q_max
    q = torch.round(x / scale).clamp(
        -q_max - 1, q_max).to(torch.int8)
    return q, scale

def dequantize(q, scale):
    return q.float() * scale

w = torch.tensor([[ -1.2, 0.5], [ -0.3, 2.1]])
q, scale = symmetric_quantize(w)
w_restored = dequantize(q, scale)
print(f"Max error: {(w - w_restored).abs().max():.4f}")
```

逐行批注：

1. $q_max = 2^{n_bits} - 1$ ：计算 INT8 的最大正值，即 $2^7 - 1 = 127$
2. $scale = x.abs().max() / q_max$ ：对称量化缩放因子 $S = \frac{\max(|x|)}{127}$
3. $torch.round(x / scale)$ ：将实值除以缩放因子后四舍五入，得到量化整数
4. $.clamp(-q_max - 1, q_max)$ ：截断到 $[-128, 127]$ 范围内
5. $.to(torch.int8)$ ：转为 INT8 类型，节省 4 倍显存（相比 FP32）
6. $dequantize$ ：反量化只需 $r = q \times S$ ，恢复近似原值

例 运行上述代码，以权重 $\begin{pmatrix} -1.2 & 0.5 \\ -0.3 & 2.1 \end{pmatrix}$ 为例：

- $S = \frac{2.1}{127} \approx 0.0165$
- 量化结果： $\begin{pmatrix} -73 & 30 \\ -18 & 127 \end{pmatrix}$
- 反量化结果： $\begin{pmatrix} -1.2045 & 0.4950 \\ -0.2970 & 2.0955 \end{pmatrix}$
- 最大误差： ≈ 0.005

INT8 量化的误差在千分之几量级，对大模型推理精度影响极小。

9 本章你将学会

1. 理解深度学习硬件基础，能解释 CPU 与 GPU 的分工以及 CUDA/cuDNN/TPU 的作用
2. 掌握剪枝的原理与流程，理解“剪枝不等于加速”的根本原因
3. 能够用对称量化公式手动计算 INT8 量化值并评估误差
4. 区分五种并行策略的显存与通信特征，能根据场景选择合适的并行方案
5. 了解 KV Cache、投机解码、连续批处理和 FlashAttention 在推理加速中的作用

10 要点速查

要点	说明
----	----

剪枝 $\neq$ 加速	细粒度稀疏需特殊硬件才真正加速
量化公式	$r = (q - Z) \times S$
量化后仍稠密	与剪枝不同，量化不改变稀疏结构
Outlier 问题	少数大权重撑大量化范围导致精度骤降
PTQ vs QAT	PTQ 成本低精度差，QAT 成本高精度好
数据并行	只提升吞吐，不降低延迟
ZeRO Stage 3	显存最省但通信最多
张量并行	通信最重，需高带宽互联
流水线气泡	需微批次 + 1F1B / DualPipe 填补
梯度累积	等效增大 batch，减少优化器步数
梯度检查点	省显存但慢 20%-30%
KV Cache	长上下文推理时是显存大头
FlashAttention	分块计算减少 HBM 读写

11 小结

本章覆盖了深度学习系统优化的全貌：从硬件基础（CPU/GPU/TPU）到三大压缩手段（剪枝让模型变稀疏、量化降精度、并行扩展规模），再到推理加速（KV Cache、投机解码、连续批处理、算子融合）。

上一章我们学习了“模型是什么”，本章回答了“如何让模型高效运行”。剪枝、量化和并行分别从稀疏化、低精度、分布式三个维度压缩模型的资源需求，而 KV Cache 和 FlashAttention 则从推理流程和算子实现层面加速计算。掌握这些技术，才能在生产环境中高效部署大模型，而非只会调用 API。

讲义基于 HPC101-2025 Day10「机器学习基础 2」课程内容编写